Reserved Particle Identifier Scheme for Decay and Special Physics Entities

VSEPR-SIM Methodology Document — Particle ID / Decay Event Subsystem

VSEPR-SIM Project

v4.0.0 — June 2025

Contents

1 Reserved Particle Identifier Scheme

To support deterministic decay handling, emitted particle bookkeeping, and future transport or defect coupling, the simulation engine defines a reserved particle identifier namespace. The convention is:

- Nonnegative identifiers correspond to ordinary matter-like species, including atoms, isotopes, molecules, or structured candidate objects.
- Negative identifiers correspond to emitted decay particles, radiation surrogates, energy stand-ins, or internal special-purpose physics entities.

Let the particle identifier be denoted by

$$ID_p \in \mathbb{Z}.$$

The initial reserved set is defined as follows:

$$\begin{aligned}
ID_p &= 0 && \to \text{generic nuclear placeholder},\\
ID_p &= -1 && \to \alpha \text{ particle},\\
ID_p &= -2 && \to \beta^- \text{ particle},\\
ID_p &= -3 && \to \gamma \text{ photon},\\
ID_p &= -4 && \to \text{generic energy packet / unresolved energy stand-in},\\
ID_p &= -5 && \to \beta^+ \text{ particle},\\
ID_p &= -6 && \to \text{electron capture bookkeeping token},\\
ID_p &= -7 && \to \text{neutron emission token},\\
ID_p &= -8 && \to \text{proton emission token},\\
ID_p &= -9 && \to \text{neutrino or unresolved loss token}.
\end{aligned}$$

1.1 Defect Token Extensions

The reserved range extends to defect-oriented tokens for lattice damage coupling:

$$\begin{aligned}
ID_p &= -10 && \to \text{vacancy defect token},\\
ID_p &= -11 && \to \text{interstitial defect token},\\
ID_p &= -12 && \to \text{defect cluster token},\\
ID_p &= -13 && \to \text{helium bubble seed},\\
ID_p &= -14 && \to \text{lattice damage packet}.
\end{aligned}$$

Future extensions may reserve additional negative ranges for transport pseudo-particles and internal engine sentinels.

2 Decay Event Transition Framework

This reserved identifier system allows nuclear events to be represented in a uniform transition framework. A decay event may be written abstractly as

$$S_{\text{parent}} \to S_{\text{daughter}} + \sum_k P_k,$$

where S_{parent} is the parent nuclear state, S_{daughter} is the daughter state, and P_k are emitted particles represented by reserved identifiers.

For alpha decay,

$$S(Z, A) \to S(Z - 2, A - 4) + P_{-1},$$

and for beta-minus decay,

$$S(Z, A) \to S(Z + 1, A) + P_{-2} + P_{-9}.$$

2.1 Energy Partitioning

Each decay event carries explicit energy partitioning:

- **Deposited fraction** — energy absorbed locally (lattice heating, recoil).
- **Transported fraction** — energy carried away by emitted particles.
- **Local damage score** — proxy for lattice displacement probability.

Default partitioning by decay mode:

Decay Mode	Deposited	Transported	Damage
Alpha (α)	0.85	0.15	0.90
Beta-minus (β^-)	0.35	0.65	0.30
Gamma (γ)	0.10	0.90	0.05

2.2 Design Purpose

The purpose of this scheme is not only categorical labeling, but also deterministic support for:

- decay chain propagation,
- local energy deposition,
- transport-ready emitted particle objects,
- defect generation and lattice damage coupling,
- universal kinetic event bookkeeping.

3 Three-Layer Architecture

The particle ID and decay event subsystem is organized into three layers:

3.1 Layer A: Identity Layer

Defines the namespace for all particle, species, isotope, and defect identifiers.

- `ParticleID` — scoped enum (`std::int32_t` backing) with constexpr queries.
- Species IDs — positive integers mapping to `ElementDescriptor` or isotope tables.
- Defect tokens — negative range $[-14, -10]$ for vacancy, interstitial, cluster, bubble, damage.

3.2 Layer B: Event Layer

Captures physics-specific event data:

- `DecayEvent` — parent/daughter IDs, emitted particle array (max 8), energy partitioning, damage score, confidence.
- `GenericTransportEvent` — source/target IDs, time, confidence (placeholder for future expansion).

3.3 Layer C: Kinetics Layer

Wraps physics events in a universal kinetic framework:

- `EventKind` — coarse classification (decay, transport, collision, adsorption, desorption, coalescence, nucleation, deposition, restructuring, phase_change).
- `KineticEventRecord` — carries EventKind, transition intensity, barrier score, environment multiplier, and a type-safe `std::variant` payload.

4 Integration with Existing Infrastructure

The particle ID system is designed to complement, not replace, existing codebase structures:

4.1 Relationship to DecayMode and DecayType

- `vsepr::nuclear::DecayMode` (in `decay_chains.hpp`) classifies *how* a nuclide decays (Alpha, BetaMinus, BetaPlus, ElectronCapture, Fission, Stable).
- `atomistic::nuclear::DecayType` (in `nuclear_stability.hpp`) predicts the dominant decay mode for a given Z.
- `vsepr::physics::ParticleID` identifies *what was emitted* as a first-class bookkeeping object.

These are orthogonal: DecayMode/DecayType answer "what kind of decay?" while ParticleID answers "what came out?"

4.2 Relationship to ElementDescriptor

`ElementDescriptor` (in `element_descriptor.hpp`) carries identity-layer data including `decay_clock` (half-life countdown), `metastable` (isomer flag), `mass_number` (A), and `isotope_label`. The new `NuclearState` struct is deliberately minimal — it carries only `species_id`, Z, and A for transition arithmetic, deferring full identity resolution to `ElementDescriptor`.

4.3 Relationship to Formation Kinetics

`atomistic::kinetic::EventType` (in `formation_kinetics.hpp`) has 28 formation-specific event types including `DECAY`, `EMISSION`, and `CAPTURE`. The new `vsepr::kinetics::EventKind` is coarser and universal — a formation-kinetics `DECAY` event maps to `EventKind::decay` at the kinetic scheduling layer.

5 C++ API Summary

5.1 Constexpr Queries on ParticleID

```cpp
constexpr bool is_reserved_particle_id(ParticleID id);
constexpr bool is_transportable(ParticleID id);
constexpr bool is_bookkeeping_only(ParticleID id);
constexpr bool is_defect_token(ParticleID id);
constexpr std::string_view to_string(ParticleID id);
```

5.2 Decay Event Builders

```cpp
DecayEvent make_alpha_decay(int32_t parent, int32_t daughter,
                            double energy_eV, double time_s,
                            double confidence);

DecayEvent make_beta_minus_decay(int32_t parent, int32_t daughter,
                                 double energy_eV, double time_s,
                                 double confidence);

DecayEvent make_gamma_decay(int32_t parent, int32_t daughter,
                            double energy_eV, double time_s,
                            double confidence);
```

5.3 Daughter-State Arithmetic

```cpp
constexpr DaughterState alpha_daughter(const NuclearState& s);
constexpr DaughterState beta_minus_daughter(const NuclearState& s);
constexpr DaughterState beta_plus_daughter(const NuclearState& s);
```

6 Plain C Legacy API

A plain C bridge is provided in `legacy/c_api/` for portable or embedded code paths:

```cpp
typedef enum ParticleID { /* 0 to -14 */ } ParticleID;

const char* particle_id_name(int id);
int particle_id_is_reserved(int id);
int particle_id_is_transportable(int id);
int particle_id_is_bookkeeping_only(int id);

typedef struct DecayEvent { /* ... */ } DecayEvent;
void decay_event_init(DecayEvent* ev);
int decay_event_add_emitted(DecayEvent* ev, int particle_id);
```

7 File Placement

File	Layer
include/physics/particle_id.hpp	A (identity)
include/physics/decay_event.hpp	B (event)
include/physics/decay_event_builder.hpp	B (event builder)
include/kinetics/kinetic_event_kind.hpp	C (kinetics)
include/kinetics/kinetic_event_record.hpp	C (kinetics)
legacy/c_api/particle_ids.h	A (C bridge)
legacy/c_api/particle_ids.c	A (C bridge)
legacy/c_api/decay_event.h	B (C bridge)
legacy/c_api/decay_event.c	B (C bridge)